

Ouvidoria Inteligente: Triagem Semântica de Manifestações Cidades

Relatório técnico · Desafio prático de NLP aplicado · Pedro Ernesto

1. Contexto e objetivo

A Ouvidoria de um município de médio porte recebe cerca de 4.000 manifestações por mês em texto livre e faz a triagem por palavras-chave. Isso gera três falhas: duplicatas com vocabulário diferente não são detectadas, temas relacionados não são agrupados e manifestações longas perdem contexto ao serem indexadas como um bloco só. Este trabalho constrói um protótipo de triagem semântica baseado em representações vetoriais, entregue em três notebooks e um app Streamlit. Este relatório resume as decisões tomadas, os resultados, as dificuldades e os aprendizados; os detalhes e gráficos completos estão nos notebooks.

2. Base de dados

O corpus tem 40 manifestações (M001 a M040) distribuídas em cinco categorias oficiais (infraestrutura: 9, saúde: 8, meio ambiente: 8, educação: 8, segurança: 7), com textos entre 121 e 799 caracteres. Seis manifestações são duplicatas semânticas de outra (15% do corpus, seis pares no gabarito *duplicatas_gabarito.json*) e cinco têm mais de 500 caracteres (M006, M015, M024, M033, M040), sendo o alvo do chunking. Como o arquivo *manifestacoes.json* não acompanhou o enunciado, ele foi reconstruído com o script *gerar_dataset.py* respeitando todas as restrições descritas, inclusive os pares M003/M017, M008/M022 e M008/M031 usados na Entrega 1. Os textos simulam manifestações reais de João Pessoa (bairros, avenidas e equipamentos públicos existentes), com variação de registro entre formal e coloquial.

3. Decisões técnicas e resultados

3.1 Entrega 1: comparação de representações

Para BoW e TF-IDF apliquei um pré-processamento leve (minúsculas, remoção de acentos e de stopwords do português) para que conectivos não inflassem a similaridade. Os embeddings recebem o texto original, já que os modelos foram treinados com linguagem natural. Comparei dois modelos multilíngues de *sentence-transformers*: *paraphrase-multilingual-MiniLM-L12-v2* (384 dimensões, rápido) e *paraphrase-multilingual-mpnet-base-v2* (768 dimensões, mais preciso). Os vetores são L2-normalizados, então produto interno = cosseno.

Par	BoW	TF-IDF	Emb. MiniLM	Emb. mpnet	Tokens em comum (após pré-processamento)
M003 × M017	0.295	0.247	0.757	0.842	av, brasil, carros, moto, perto
M008 × M022	0.123	0.122	0.609	0.734	atendimento, geisel
M008 × M031	0.067	0.021	0.228	0.388	bairro

A leitura é direta: as representações esparsas só enxergam sobreposição lexical. No par M008 × M022 os dois textos descrevem o mesmo posto sem médico, mas com palavras diferentes ("posto de saúde" vs. "PSF", "sem médico" vs. "falta atendimento"), e a similaridade esparsa depende de poucos tokens coincidentes; no par M008 × M031 ela é próxima de zero pelo motivo errado (só a palavra genérica "bairro" em comum, não porque o modelo entenda que são assuntos diferentes). Os embeddings colocam os dois pares duplicados bem acima do par sem relação (MiniLM: 0.76 e 0.61 contra 0.23; mpnet: 0.84 e 0.73 contra 0.39), que é o comportamento desejado para triagem. Ainda assim, M008 × M022 fica abaixo do 0,85 sugerido no enunciado nos dois modelos, e no MiniLM M022 é apenas o segundo vizinho de M008, atrás de M002; esse par reaparece como falso negativo na Entrega 2. Na análise global (similaridade média intra vs. inter categoria), a razão intra/inter foi de 2.33 para BoW, 2.94 para TF-IDF, 1.70 para MiniLM e 1.54 para mpnet. A razão das esparsas parece alta, mas vem de um piso quase zero (a maioria dos pares não compartilha token nenhum); nos embeddings a matriz ordenada por categoria exhibe blocos visíveis na diagonal, com mistura entre infraestrutura e segurança.

Limitações. BoW ignora ordem e semântica e é sensível a tamanho. TF-IDF corrige o peso de termos genéricos, mas continua lexical e, num corpus de 40 documentos, o IDF é instável. Embeddings resolvem paráfrase e sinonímia, mas custam mais, dependem de modelo pré-treinado, são opacos (não dá para apontar "a palavra X causou o agrupamento"), representam mal nomes locais e siglas municipais e truncam textos acima de 128 tokens, o que motiva a Entrega 3.

3.2 Entrega 2: detecção de duplicatas

A função *detectar_duplicatas(textos, limiar=0.85)* codifica os textos com o MiniLM, calcula a matriz de similaridade e devolve os pares acima do limiar, com opção de reaproveitar embeddings pré-calculados. O

heatmap 40×40 (triângulo inferior) está no notebook e em *resultados/heatmap_similaridade.png*. Para justificar o limiar, olhei a distribuição das 780 similaridades entre pares e varri limiares de 0,50 a 0,95, medindo precisão, recall e F1 contra o gabarito. A distribuição tem média 0.284 e percentis p90 = 0.480, p95 = 0.550 e p99 = 0.665.

Limiar	Precisão	Recall	F1	VP	FP	FN
0,85 (enunciado)	1.00	0.33	0.50	2	0	4
0.70 (escolhido, F1 máx.)	0.80	0.67	0.73	4	1	2

Justificativa do limiar. O F1 máximo (0.73) ocorre em um único ponto da grade, 0.70, então a regra de desempate que adotei (preferir o maior limiar entre os empatados) não chegou a ser acionada. A preferência pelo limiar mais alto, em caso de empate, é por ser a opção mais conservadora: para a Ouvidoria, fundir indevidamente duas manifestações diferentes pode deixar um cidadão sem resposta, enquanto uma duplicata não detectada custa apenas retrabalho. O valor 0,85 do enunciado é comparado na tabela: com ele o MiniLM detecta só 2 dos 6 pares do gabarito (recall 0.33), sem falsos positivos, o que é restritivo demais para a escala de similaridade deste modelo. Também avaliei o limiar dinâmico por percentil (dica do enunciado): o p90 pressupõe que 10% dos pares são duplicatas, muito acima da realidade (6 em 780, 0,8%), então o p99 é o percentil adequado. A vantagem do percentil é se adaptar automaticamente à escala de similaridade de cada modelo, que muda entre MiniLM e mpnet, como a comparação no notebook mostra.

Erros. Falsos positivos: M028×M031. Falsos negativos: M008×M022, M019×M038. O falso positivo é o caso típico de manifestações sobre o mesmo tipo de problema em lugares diferentes (iluminação pública na rua Manoel Deodato e na praça da Torre): o embedding captura "o que" e não "onde". Uma correção prática é combinar o score semântico com um filtro de entidade (bairro/rua) via regex ou NER. Os falsos negativos são textos curtos e de tamanho parecido, com similaridades 0.61 e 0.69 (acima do percentil 95, mas abaixo do limiar); o que muda entre as versões é o vocabulário (posto de saúde/PSF, médico/profissional; assaltada/roubos, ponto/parada) e o ponto de vista, um relato pessoal de um lado e a descrição de um problema recorrente do outro. Para esses casos o ganho vem de um modelo maior (o mpnet dá 0.73 a M008 × M022) ou de um limiar menor combinado com o filtro de local, e não do chunking, já que nenhum dos dois pares envolve texto longo.

3.3 Entrega 3: chunking das manifestações longas

O MiniLM trunca a entrada em 128 tokens; as cinco manifestações longas passam desse limite (entre 190 e 200 tokens), então o final do texto (que costuma conter o pedido concreto) seria ignorado. Usei o *RecursiveCharacterTextSplitter* com três configurações: A (200, 0), B (200, 60) e C (350, 80). Para cada uma medi a coesão consecutiva (cosseno entre chunks vizinhos), a fidelidade média (cosseno entre cada chunk e o texto completo) e a cobertura (cosseno do melhor chunk com o texto completo), além da silhueta dos chunks agrupados pela manifestação de origem.

Config	Chunks (total)	Coesão consecutiva	Fidelidade média	Cobertura	Silhueta
A (200, 0)	20	0.458	0.692	0.887	0.195
B (200, 60)	29	0.562	0.655	0.887	0.203
C (350, 80)	15	0.582	0.747	0.963	0.273

Overlap e coesão. Entre A e B a única diferença é o overlap, e a coesão consecutiva sobe: chunks vizinhos passam a compartilhar ~60 caracteres idênticos, o que puxa seus vetores um para o outro e reduz o risco de uma frase importante ficar cortada ao meio sem estar inteira em nenhum chunk. O custo é redundância (mais chunks, trechos contados duas vezes no ranking da busca); não testei overlaps maiores, e a expectativa é que acima de ~30% do chunk_size o ganho diminua. **Configuração recomendada: C.** Chunks de ~350 caracteres ficam abaixo do limite de tokens (nada é truncado) e ainda contêm sujeito + problema + local; C obteve a maior fidelidade e cobertura. B é melhor para busca por detalhe pontual ("falta de insulina" cai exatamente num chunk). Em M006, por exemplo, A corta a frase dos atendentes ("dizem que o estoque não chega e que | devemos voltar em uma semana") entre dois chunks, e o overlap de B a devolve inteira no chunk seguinte; em C a lista de remédios (chunk 0) e a agenda de consultas com o pedido final (chunk 2) ficam em chunks distintos e inteiros. **Espaço 2D.** Nos gráficos PCA/t-SNE os chunks da mesma manifestação formam grupos, mas não compactos: os primeiros chunks (que apresentam o problema) ficam perto do centróide, e os últimos (o pedido genérico à prefeitura) se aproximam dos chunks finais de outras manifestações. Chunks maiores (C) agrupam melhor.

3.4 Entrega 4: app Streamlit

app_ouvidoria.py tem as quatro abas pedidas. A sidebar seleciona o modelo (MiniLM ou mpnet) e o top-k. O modelo fica em *st.cache_resource* e os embeddings da base em *st.cache_data*, chaveados pelo nome do

modelo, então trocar de modelo recalcula uma vez e depois é instantâneo. Na Busca Semântica, além da lista com score colorido (verde/amarelo/vermelho), o app resume o resultado em uma frase ("provável duplicata", "há relacionadas", "problema novo"), oferece exemplos prontos e filtro por categoria. A Base Completa tem métricas do corpus, tabela completa, heatmap interativo (Plotly, com hover mostrando o par e o valor) e a lista de candidatos a duplicata para um limiar ajustável. O Espaço Vetorial permite alternar PCA/t-SNE (com perplexidade ajustável), mostra a silhueta por categoria e traz o comentário pedido: saúde e educação formam clusters coesos; infraestrutura e segurança se misturam (iluminação pública aparece nas duas); meio ambiente se espalha entre infraestrutura e saúde; as manifestações longas ficam no centro do mapa por tratarem de vários temas. A aba Chunking aceita texto colado, permite escolher estratégia (Recursive ou Character) e parâmetros, e exibe cada chunk com sua fidelidade ao documento, as primeiras dimensões dos embeddings e a matriz de similaridade entre chunks.

4. Dificuldades

Dataset ausente. O JSON não veio com o enunciado; reconstruí um corpus fiel à especificação, com gabarito de duplicatas explícito, o que na verdade tornou a avaliação da Entrega 2 mais rigorosa. **Overlap que desaparece.** Minha primeira versão do chunking usava separadores por frase (". ", ", ") para cortes mais naturais. Descobri que o splitter só reaproveita pedaços inteiros para formar o overlap; como as frases têm 80 a 150 caracteres, um overlap de 60 não cabia em nenhuma e o overlap efetivo caía para ~2 caracteres, tornando A e B idênticas. A solução foi voltar aos separadores padrão (nível de palavra) e registrar o achado no notebook. **Limiar não transferível.** A escala de similaridade muda entre modelos; o 0,85 do enunciado não significa a mesma coisa para MiniLM e mpnet, daí a importância da varredura de limiares e do limiar por percentil. **t-SNE em amostra pequena.** Com 40 pontos (ou 15 a 30 chunks) a perplexidade precisa ser baixa e o resultado varia com a semente; fixei *random_state* e tratei o t-SNE como leitura qualitativa, mantendo o PCA ao lado. **Tipos do pandas.** Colunas de string com backend Arrow não aceitam indexação 2D no NumPy; foi preciso converter explicitamente para *object* antes de construir a máscara de categorias.

5. Aprendizados

(1) Representações esparsas falham exatamente no caso que motiva o projeto, a paráfrase; embeddings resolvem isso, mas trazem um novo tipo de erro: juntam o mesmo problema em lugares diferentes, porque capturam o assunto e não a entidade. O sistema final deve combinar os dois (score semântico + filtro lexical de local). (2) Um limiar não é uma constante: é uma decisão de negócio que depende do custo de cada erro e da escala do modelo, e deve ser calibrada com um gabarito e revisada a cada troca de modelo. (3) Chunking não é só "caber no modelo": o tamanho do chunk define a granularidade da busca, e a métrica de fidelidade/cobertura ajuda a escolher sem depender do olho. (4) Detalhes de implementação (separadores do splitter, cache no Streamlit, tipos do pandas) mudam resultados e merecem ser documentados, já que o código será lido por outra pessoa da equipe.

6. Como executar

`pip install -r requirements.txt`, depois rodar os notebooks na ordem (1, 2, 3), que gravam seus resultados em `resultados/`; `streamlit run app_ouvidoria.py` abre o app; `python gerar_relatorio.py` regenera este PDF com os números atualizados. Os modelos são baixados do Hugging Face na primeira execução.